

Contents

CampusEats – Assignment 5	1
Team Members	1
1. CampusEats Method Map	1
2. Headers Table	2
3. Safe-Retry Plan (from C4)	2
4. Full HTTP Exchange	3
5. Question 1 — Three endpoints: method, success status, key response header, why it matters . .	3
6. Question 2 — Safe / idempotent classification	4
7. Question 3 — ETag, 304, and 412	5
8. Question 4 — 422 vs 400	5
Appendix — Correctness review of Parts A–C (as submitted by the group)	6

CampusEats – Assignment 5

Team ID: 21

Team Members

1. Roll No: 20251651098 Name: Uday Kumar Gupta
2. Roll No: 20251651059 Name: Mukesh
3. Roll No: 20251651046 Name: Ritesh
4. Roll No: 20251651105 Name: Yash
5. Roll No: 20251651052 Name: Koushik

1. CampusEats Method Map

Action	Method	URL	Purpose
List vendors	GET	/vendors	Read vendor list
View vendor	GET	/vendors/{vendorId}	Read one vendor
List menu items	GET	/vendors/{vendorId}/menu-items	Read/filter menu items
View menu item	GET	/menu-items/{itemId}	Read one menu item
Create order	POST	/orders	Create a new order
View order	GET	/orders/{orderId}	Read one order
Replace order	PUT	/orders/{orderId}	Replace complete order representation
Delete order	DELETE	/orders/{orderId}	Remove/cancel an order where permitted
Create payment	POST	/orders/{orderId}/payments	Create a payment resource
Track order	GET	/orders/{orderId}/status	Read current order status
Checkout an order	POST	/orders/{orderId}/checkout	Non-CRUD action modeled as a sub-resource
Cancel an order	POST	/orders/{orderId}/cancel	Non-CRUD action modeled as a sub-resource

Action	Method	URL	Purpose
Set vendor availability	POST	/vendors/{vendorId}/availability	Not a CRUD action modeled as a sub-resource

Part D verification note: the curl transcript below exercises the same HTTP mechanics (create with Idempotency-Key, conditional GET/PUT with ETag, and Authorization) against a small **restaurants** resource, because the Part D task sheet specifies **/restaurants** endpoints explicitly. The underlying method/response-code rules are identical to the CampusEats **/orders** design above.

2. Headers Table

Endpoint	Request Headers	Response Headers
GET /restaurants/1	Accept, If-None-Match	Content-Type, ETag, Cache-Control, CORS, security
POST /restaurants	Content-Type, Accept, Idempotency-Key	Content-Type, Location, ETag, CORS, security
PUT /restaurants/1	Content-Type, If-Match	Content-Type, ETag, CORS, security
DELETE /restaurants/1	Authorization	CORS, security
OPTIONS /restaurants/1	Origin, Access-Control-Request-Method	Allow, CORS headers
GET /orders/1	Authorization	Content-Type, CORS, security

3. Safe-Retry Plan (from C4)

Endpoint	Safe?	Idempotent?	Retry-Safety Mechanism
GET /vendors	Yes	Yes	Naturally retry-safe (read-only)
GET /vendors/{vendorId}	Yes	Yes	Naturally retry-safe (read-only)
GET /vendors/{vendorId}/menuitems	Yes	Yes	Naturally retry-safe (read-only)
GET /orders/{orderId}	Yes	Yes	Naturally retry-safe; supports conditional GET (ETag / If-None-Match)
GET /orders/{orderId}/status	Yes	Yes	Naturally retry-safe (read-only)
PUT /orders/{orderId}	No	Yes	Naturally idempotent by definition; additionally protected against lost updates with If-Match / ETag (412 on stale write)

Endpoint	Safe?	Idempotent?	Retry-Safety Mechanism
DELETE /orders/{orderId}	No	Yes	Naturally idempotent (repeated delete leaves resource absent)
POST /orders	No	No	Made retry-safe explicitly with an Idempotency-Key header; server caches the first result per key and replays it on retry instead of creating a duplicate
POST /or- ders/{orderId}/checkout	No	No	Made retry-safe explicitly with an Idempotency-Key header, same mechanism as POST /orders

4. Full HTTP Exchange

Request:

```
POST /orders HTTP/1.1
Host: campuseats.example
Authorization: Bearer <token>
Content-Type: application/json
Accept: application/json
Idempotency-Key: order-2026-001
```

```
{"vendorId": "v12", "items": [{"menuItemId": "m45", "quantity": 2}], "paymentMethod": "UPI"}
```

Response:

```
HTTP/1.1 201 Created
Location: /orders/987
Content-Type: application/json
Cache-Control: no-store
```

```
{"orderId": "987", "status": "PLACED", "vendorId": "v12", "items": [{"menuItemId": "m45", "quantity": 2}]}
```

5. Question 1 — Three endpoints: method, success status, key response header, why it matters

Endpoint	HTTP Method	Success Status	Most Important Response Header	Why It Matters
POST /orders	POST	201 Created	Location: /orders/{orderId}	Tells the client exactly where the newly created order now lives, so it can immediately GET/PUT/DELETE that resource without guessing or re-parsing the body for an ID.
GET /orders/{orderId}	GET	200 OK	ETag: "order-vN"	Gives the client a validator for the current version of the resource. It is what makes conditional GET (If-None-Match → 304) and safe concurrent updates (If-Match → 412) possible.
DELETE /orders/{orderId}	DELETE	204 No Content	<i>(no body — absence of Content-Type is itself the signal)</i>	Confirms the deletion succeeded and explicitly tells the client not to expect a response body, avoiding wasted parsing and ambiguous “empty JSON” responses.

6. Question 2 — Safe / idempotent classification

Safe endpoints (do not change server state): - GET /vendors - GET /vendors/{vendorId} - GET /vendors/{vendorId}/menuitems - GET /menu-items/{itemId} - GET /orders/{orderId} - GET /orders/{orderId}/status

Idempotent endpoints (repeating the request produces the same end state): - All the safe GET endpoints above (safe implies idempotent) - PUT /orders/{orderId} — replacing a resource with the same representation twice leaves it in the same final state - DELETE /orders/{orderId} — deleting an already-deleted resource leaves it absent either way

Endpoint that is neither safe nor idempotent: - POST /orders (and similarly POST /orders/{orderId}/checkout) — each call is intended to create a new resource / trigger processing, so a naive retry after a timeout could create a second order or trigger payment twice.

How it was made retry-safe: POST /orders was made retry-safe by requiring an Idempotency-Key header on the request. The server stores the key together with the result (status code + body + headers) of the *first* successful request. If the same key arrives again — for example because the client’s connection timed out and it retried — the server does not repeat the creation logic; it simply looks up the stored result for that

key and returns the exact same response (same order ID, same status 201, same Location), so no duplicate order is ever created.

7. Question 3 — ETag, 304, and 412

One ETag:

ETag: "restaurant-1-v1"

Request returning 304 (Not Modified):

GET /restaurants/1 HTTP/1.1

Host: localhost:5000

If-None-Match: "restaurant-1-v1"

HTTP/1.1 304 Not Modified

ETag: "restaurant-1-v1"

Cache-Control: private, max-age=60

(captured live in curl-transcript.txt, section 3)

Write returning 412 (Precondition Failed):

PUT /restaurants/1 HTTP/1.1

Host: localhost:5000

Content-Type: application/json

If-Match: "old-etag"

{"name": "New Name"}

HTTP/1.1 412 Precondition Failed

ETag: "restaurant-1-v1"

{"error": "Precondition Failed", "message": "Resource has changed; refresh before updating."}

(captured live in curl-transcript.txt, section 4)

What each prevents/saves: - **304 (If-None-Match):** Saves bandwidth and processing. If the client's cached copy is still current, the server sends only headers and no body, so the same JSON payload is not re-transmitted for no reason. - **412 (If-Match):** Prevents the "lost update" problem. If another client (or another tab/request) has already changed the resource since this client last read it, the ETags will no longer match, and the server refuses to blindly overwrite the newer version with stale data — protecting against silently discarding someone else's update.

8. Question 4 — 422 vs 400

Exact request producing 400 Bad Request (malformed JSON — the body cannot even be parsed):

POST /restaurants HTTP/1.1

Host: localhost:5000

Content-Type: application/json

{"name":

(captured live in curl-transcript.txt, section 5 → HTTP/1.1 400 Bad Request)

Exact request producing 422 Unprocessable Content (syntactically valid JSON that violates a business/domain rule):

```
POST /orders HTTP/1.1
Host: campuseats.example
Content-Type: application/json
```

```
{"vendorId": "v12", "items": [{"menuItem": "m45", "quantity": -2}], "paymentMethod": "UPI"}
```

Expected:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json
```

```
{"error": "Unprocessable Content", "message": "quantity must be a positive integer"}
```

The difference: - **400 Bad Request** means the request is malformed at the syntax/transport level — the server literally cannot parse the body (broken JSON, missing required structural elements). The request never even reaches domain validation. - **422 Unprocessable Content** means the request is syntactically well-formed (valid JSON, correct Content-Type) but fails a semantic or business rule once the server understands it — e.g. a negative quantity, an unknown vendorId, or a menu item that isn't available. The JSON parser succeeds; the domain/business logic is what rejects it.

Appendix — Correctness review of Parts A–C (as submitted by the group)

Parts A, B, and C are structurally correct and match standard REST semantics: - **A1–A6:** Method map, non-CRUD modeling via sub-resources, and the safe/idempotent table are all correct. **POST /orders** and **POST /orders/{orderId}/checkout** are correctly flagged as neither safe nor idempotent. - **B1–B7:** Status codes, headers (**Location**, **Etag**, **Cache-Control**, **Authorization**, rate-limit headers, **CORS**, security headers) are all correctly matched to their situations. - **C1–C3:** The ETag-based conditional GET (304), conditional write (412), and Idempotency-Key mechanisms are described correctly and match the working Express.js implementation shown in Part C, section 5, whose live curl output already demonstrates a real 200 OK with **Etag: "order-v1"**.

One inconsistency to flag with your groupmate: Part D's own instructions use a **/restaurants** resource (**POST /restaurants**, **GET/PUT /restaurants/{id}**), while Parts A–C build out **/vendors** and **/orders** for CampusEats. This is a mismatch in the assignment materials themselves, not an error in your teammate's reasoning — the underlying HTTP mechanics (status codes, Etag/If-Match/If-None-Match, Idempotency-Key) are identical either way. For Part D below, the curl transcript follows the task sheet literally and tests **/restaurants** (plus **/orders/1** for the 401 case, as instructed), so the deliverable matches exactly what was assigned.